The Final Exam (total 15 points, due at 9:00 on 27.02.26)

Working in groups and/or using any kind of AI is strictly prohibited! If detected, the exam result will be annulled!

Problem 1 (2 points)

You need to write a kernel that operates on an image of size 3840 x 2160 pixels. Each thread processes one pixel by forming a convolution of a rectangular 9x7 mask centered at the pixel with the appropriate number of surrounding pixels (assume that each pixel is characterized by four floating point parameters), with the “ghost” pixels not contributing to the result. You would like your threadblocks to be square, the mask to be separate for each parameter and to be stored in shared memory. Assuming your device has CUDA compute capability 8.6 and you select the grid dimensions and block dimensions of your kernel to ensure maximum occupancy

Questions:

1) How many warps will have control divergence?
2) Assuming that the algorithm bottleneck is caused by global memory operations, what would be the speedup factor if the pixel data get shifted to the shared memory buffer?

Solution:

1.
Relevant parameters are

Maximum number of resident blocks per multiprocessor: 16

Maximum number of resident threads per multiprocessor: 1536

Maximum amount of shared memory per multiprocessor: 100 KiB

Maximum number of threads per block: 1024

Technically, for square blocks maximum occupancy can be reached for only one block configuration with 16x16 threads, which results in 6 resident blocks per SM. Shared memory does not impose any limitations in this case, since even for 6 blocks it will result in 63 x 4 x 6 x sizeof(float) = 6 KiB < 100 KiB. If handling the out-of-boundary issues are made through the ghost cells (as the problem description suggests) as opposed to conditional statements, the only warp divergence that can occur can be caused by the conditional statement checking if a thread is within the x image boundary. However, this does not happen since 3840 is evenly divisible by 16. In this case, there will be no warps with control divergence. If contributions from the ghost cells are handled by conditional statements, then one can estimate the number of divergent warps from adding (2160/2)x2 for the warps on the left and right boundaries and ((3840/16)-2)x2, since for the bottom and top boundaries there will be two divergent warps (because the convolution stencil has 7 points), but we have already counted two warps on each of the boundaries when considering left or right boundaries. In total, this yields 2636 warps.

2.

Assuming that cache units have no effect and neglecting the small corrections at the boundaries, the common global memory operations for both versions will include reading the mask (63x4=252 operations) and storing the output (16x16x4=1024 operations). As far as the convolution is concerned, reading pixel data from global memory only will result in approximately 3840x2160x63x4 global memory operations (since for each convolution centered at a given pixel one has to fetch all neighbor data). If one puts the pixel data into shared memory, one has to consider the threadblock partitions. In this case, each threadblock will put a tile of global pixel data equal to (16+6)x(16+8)x4 = 2112 floating point numbers (the additional numbers result from the need to fetch the data elements beyond the tile boundaries) and store them into shared memory buffer, whereupon it will be used with a negligible memory overhead. The number of such blocks is (3840/16)x(2160/16) = 240x135, hence the total number of global memory operations will be approximately 2112x240x135. Therefore, the speedup is approximately (3840x2160x63x4+1276*240*135)/((2112+1276)*240*135) = 19.4. Of course, in reality the cache units will significantly reduce this speedup.

Problem 2 (5 points)

Assume that a student wrote the following simple kernel with 1d execution configuration (each thread calculates one element of the final vector P) for multiplication of a square matrix and a corresponding vector, where Width is not a multiple of blockDim.x:

```
__global__ void MatrixMulKernel(float* M, float* N, float *P, int Width)
{

  float Pvalue;
  int Row = blockIdx.x*blockDim.x+threadIdx.x;
  for (int k = 0; k <= Width; k++) {Pvalue += M[Row*Width+k] * N[k]}

  P[Row] = Pvalue;
}
```

Questions:

1) Find all errors in the code
2) How would you implement data reuse by utilizing a shared memory buffer (provide the modified code)? Assuming that the cache hierarchy does not play a significant role, by what factor will the effective global memory bandwidth increase after this optimization?
3) Will the “shared memory” version from 2) be compute- or memory-bound (assume that you have a RTX 3090 GPU)?
4) Suppose that you want to replace the “for” loop with the parallel reduction algorithm based on the binary tree concept. How should the thread-data mapping and the execution configuration be modified to get the best performance? Give the corresponding code modifications.

Solution:

1.

- Pvalue has to be initialized

- in the for loop the index k has to run from 0 to Width-1

- There is a semicolon missing inside the curly brackets

- There is a missing conditional statement checking if the Row index is inside the matrix size in the x dimension.

The corrected kernel should look like:

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
  int Row = blockIdx.x*blockDim.x+threadIdx.x;
  If (Row<Width){
    float Pvalue = 0;
    for (int k = 0; k < Width; k++) {Pvalue += M[Row*Width+k] * N[k];}
    P[Row] = Pvalue;
    }
}
```

2.

Note that the same input vector data (stored in the N array) is used by all threads. If one reads it once from the global memory and store it in the shared memory, we will benefit from the data re-use. The corresponding kernel should look something like

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
  extern __shared__ float shBuff[];  // since Width is not known at the compile time
  // read the input vector from global memory to the shared memory buffer
  int k = threadIdx.x;
  while (k<Width){
    shBuff[k] = N[k];
    k += blockDim.x;
  }
```

```
  __syncthreads();


  int Row = blockIdx.x*blockDim.x+threadIdx.x;
  If (Row<Width){
    float Pvalue = 0;
    for (k = 0; k < Width; k++) {Pvalue += M[Row*Width+k] * shBuff[k];}
    P[Row] = Pvalue;
    }
}
```

Note that the kernel uses one-dimensional execution configuration and the most bandwidth-intensive part of it occurs in the calculation of scalar products between the rows of the matrix M and the input vector N (in the original kernel it is the following line: for (k = 0; k < Width; k++) {Pvalue += M[Row*Width+k] * N[k];}). Note that such a thread arrangement is not the most effective one, since it does not ensure the global memory coalescence for reading the matrix M, because the adjacent threads read elements of the matrix M on the same column. Such a strided pattern of reading the matrix M means that the corresponding effective bandwidth will be terrible – it will be 1/8 of the peak bandwidth for a large Width assuming the reads are performed by segments of 32 B (with the L1 cache off). As far as the input vector N is concerned, each thread in a warp will most likely read the same element N[k] during each iteration of the “for” loop, which will result in the broadcast of that element to all the threads in the warp. If the L1 cache is turned off, the effective bandwidth of reading N is again 1/8 of the peak bandwidth. Let us show that the total resulting bandwidth will be also 1/8 of the peak bandwidth. Denoting the peak bandwidth as BWg, amount of useful global data read in each reading transaction of an element M[Row*Width+k] as DM, total amount of global data read during the reading transaction of an element M[Row*Width+k] as DMtot, and the similar values for the reading transaction of an element N[k] as DN and DNtot, and pointing out that for each reading transaction of an element of M one needs a reading transaction of an element of N, so that DM = DN, and that DM/DMtot = DN/DNtot = 1/8 (because for each 32 B transaction only 4 B is used), one can write for the total effective bandwidth

Dtot = total amount of data / total execution time

= (DM + DN)/((DMtot/BWg) + (DNtot/BWg)) = BWg / 8.

Now, assuming that the initial loading of the input vector N into the shared memory buffer results in a negligible number of reads compared to the number of reads of the N vector elements during the calculation of the scalar products in the “for (k = 0; k < Width; k++) {Pvalue += M[Row*Width+k] * shBuff[k];} “ line of the kernel, denoting the effective bandwidth of the shared memory as BWs and noting that BWs >> BWg, one can calculate the effective bandwidth if the shared memory is used as

Dtot = total amount of data / total execution time

= (DM + DN)/((DMtot/BWg) + (DNtot/BWs)) = BWg / 4 (assuming BWs is infinite)

Therefore, the effective bandwidth will grow approximately by a factor of 2 if the data re-use is utilized.

3.

In this case, the compute-to-global memory access ratio in the loop will be equal to 5, if one takes into account the addition operation needed to calculate the array index and the loop variable's increment. This means that this kernel will still be memory-bound.

4.

A possible variant would be to let each threadblock calculate a single value of the output vector. In this case one can use the parallel reduction algorithm to obtain this value by using all threads in cooperation, i.e. the execution configuration would be <<< ne, nt >>> , where ne is the number of elements of the input vector (also equal to each of the matrix dimensions) and nt is the number of threads per threadblock optimal for the parallel reduction algorithm (for instance, 256).

Problem 3 (1 point)

Assume you have a new RTX 3090 with the DRAM system having peak bandwidth of 936 GB/s. Assume a thread block size of 1024, warp size of 32, and that A is a floating point precision array in the global memory.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
float temp = A[9*i];
```

What is the maximal global memory effective bandwidth we can hope to achieve in the following access to A with the L1 cache disabled?

Solution:

If the L1 cache is disabled, the memory system uses either 32, 64, or 128 byte segments for the global memory reads, which are multiples of the L2 cache line equal to 32 B. For such a strided read pattern the global memory system will use the 32 B segments (or 8 floating point numbers) to read the data. This means that in such a pattern out of 8 floating point numbers (32 B) read each time only 1 (or 4 B) will be used. That means that the effective bandwidth is 1/8 of the peak bandwidth, i.e., 117 GB/s.

Problem 4 (1 point)

Starting from the Fermi architecture, NVIDIA engineers have decreased latency of performing atomic functions by executing them in the L2 cache memory, provided that the needed data is there. Should it not be the case, the data is moved to the cache and then the atomic function is performed.

For a processor that supports atomic operations in L2 cache, assume that each atomic operation takes 4 ns to complete in L2 cache and 100 ns to complete in DRAM. Assume that 35% of the atomic operations hit the L2 cache. What is the approximate throughput for atomic operations on the same global memory variable?

Solution:

The average time it takes the atomic operation to execute is 4*0.35+100*0.65 = 66.4 ns. This means that the throughput (the average number of atomic operations executed per second) will be 1 / (66.4 x 10-9) = 1.506 x 10^7 atomic operations / second.

Problem 5 (3 points)

The following scalar product code tests your understanding of the basic CUDA model.The following code computes 2,048 dot products, each of which is calculated from a pair of 256-element vectors. Use the code to answer the following questions. Assume the compute capability of your device to be 8.0.

```
1  #define  VECTOR_N  2048
2  #define  ELEMENT_N  256
3  const int  DATA_N = VECTOR_N*ELEMENT_N;
4  const int  DATA_SZ = DATA_N*sizeof(float);
5  const int  RESULT_SZ = VECTOR_N*sizeof(float);
. . .
6  float *d_A, *d_B, *d_C;
. . .
7  cudaMalloc((void **)&d_A, DATA_SZ);
8  cudaMalloc((void **)&d_B, DATA_SZ);
9  cudaMalloc((void **)&d_C, RESULT_SZ);
. . .
10  scalarProd <<< VECTOR_N, ELEMENT_N >>>(d_C, d_A, d_B, ELEMENT_N);
11
12  __global__  void
13  scalarProd(float *d_C, float *d_A, float *d_B, int ElementN)
```

```
14 {
15   __shared__  float  accumResult[ELEMENT_N];
16   //Current vectors bases
17   float *A = d_A + ElementN*blockIdx.x;
18   float *B = d_B + ElementN*blockIdx.x;
19   int  tx = threadIdx.x;
20
21   accumResult[tx] = A[tx] * B[tx];
22
23   for  (int  stride = ElementN / 2;  stride > 0;  stride >>= 1)
24   {
25     __syncthreads();
26     if  (tx < stride)
27       accumResult[tx] += accumResult[stride + tx];
28   }
30   d_C[blockIdx.x]  = accumResult[0];
31 }
```

Questions:

1) How many accesses to shared memory are done for each block?
2) List the source code lines, if any, that cause shared memory bank conflicts.
3) Identify an opportunity to reduce the bandwidth requirement on the global memory. How would you achieve this? How many accesses can you eliminate?

Solution:

1.

Number of accesses to shared memory during the shared memory initialization (line 21) is ELEMENT_N (256), total number of accesses (reads and writes) to the shared memory during the reduction (line 27) is 3*(ELEMENT_N-1) = 3*255=765, and number of accesses to shared memory done in writing the result from the shared memory (line 30) is ELEMENT_N (256). In total, this makes 256 + 765 + 256 = 1277 accesses to the shared memory.

2.

There are no bank conflicts.

3.

Line 30 contains redundant writes, since each of the threads in a block writes the result common to the entire block to the d_C[blockIdx.x] variable. It is sufficient that only “the master thread” does the writing. The line 30 should thus look like

if (threadIdx.x==0) d_C[blockIdx.x] = accumResult[0];

After this correction, the number of writes to the global memory in each block will be reduced from ELEMENT_N (256) to a single one (1). Therefore, one eliminates 255 writes to the global memory by this substitution.

Problem 6 (2 points)

Consider the scan (prefix sum) kernel based on the algorithm of Harris (actually, it was originally proposed by Blelloch) discussed in the class (e.g., Lecture 9, Slide 22)

Questions:

1) What would happen if you removed the last __synchthreads() in the algorithm?
2) Assume that you have 2048 elements in each section and warp size is 32, how many warps in each block will have control divergence during the reduction tree phase iteration where stride is 16?

Solution:

1.

The last __syncthreads() (the one that goes after the reverse tree section) is actually redundant: during the last step of the reverse tree the shared memory array values that are updated are used by the same threads immediately afterwards in order to update the global memory array (note that during the last iteration of the “for” loop preceding the last __syncthreads all the threads will be active, provided that n = 2*blockDim.x, as it is tacitly assumed in this implementation). Since threads use the same values that they have just updated, there is no risk of getting inconsistent data. In some other situation, where threads try to fetch shared memory data expected to be updated by other threads, one always need to use __syncthreads() to ensure the data consistency.

2.

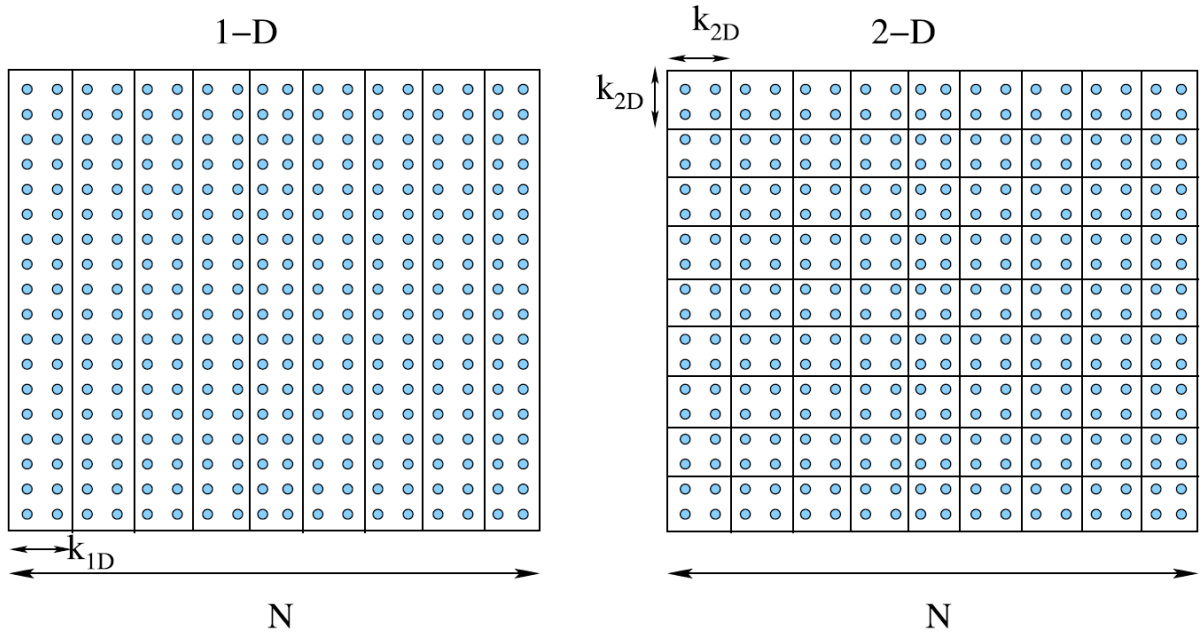
None. In the example that we considered in class, the conditional that gives the control divergence is “if (tid<d)” with d the level of the inverse reduction binary tree (see the kernel for the prefix scan in the lecture materials). When the stride (given by the “offset” in the same kernel) is equal to 16, d is equal to 64 (note that in the reduction part of the algorithm the offset variable starts with 1 and then is increased by a factor of 2 at the end of each reduction tree iteration (which corresponds to a level of the reduction tree), whereas the d variable starts with the half of the number of elements, which is 1024 in our case, and is reduced by a

factor of 2 at the end of each reduction tree iteration). Thus, none of the warps will have control divergence.

Problem 7 (1 point)

Suppose that you need to solve a huge heat diffusion problem using a stencil-based method resulting from finite differencing (see slide 25 in lecture 6), and the N x N cell data do not fit into a single GPU. Rather, you have P gpus at your disposal and you would like to use all of them. Assuming that

1) N is evenly divisible by P
2) Neglecting the intra-node communication and assuming that the inter-node communication is full-duplex (sending and receiving data can be made simultaneously), whereas each inter-node data communication has communication startup latency t_{start} , and the communication cost per value per data transaction between two gpus t_{comm} .
3) That the processing cost per single update for a single cell is t_{comp}



, estimate the t_{start}/ t_{comm} ratio, when the 1D problem decomposition (see the left figure, here each gpu node has k_{1D} = N/P cells in x direction and N cells in the y direction) leads to a better algorithm performance than the 2D problem decomposition (see the right figure, assume that here $k_{2D}^2=N^2/P$, since the work should be evenly divided between P gpus).

Solution:

The heat diffusion contains the Laplacian operator, which, upon discretization, requires communication of four values from neighboring cells (north, south, west and east). If the cells are inside the same node, one can assume fast communications and take into account only the latency and communication costs associated with inter-node data transfers.

For **1D** problem decomposition, the time spent per node for computations and communications is

$T_{comp,1D} = k_{1D}\, N\, t_{comp} = (N^2/P)\, t_{comp}$

and

$T_{comm,1D} = 2\,(t_{start} + N\, t_{comm})$

, respectively. The communication time is calculated based on the need for each node (except the boundary nodes) to send and receive two batches of N values, whereas full duplex enables a simultaneous send and receive (hence one count only sends, receives come at no cost). For simplicity, one can neglect the corrections at the boundary nodes (which are small for a sufficiently large P).

Following the same logic, for **2D** problem decomposition, the time spent per node for computations and communications is

$T_{comp,2D} = k_{2D}^2\, t_{comp} = (N^2/P)\, t_{comp}$

and

$T_{comm,2D} = 4\,(t_{start} + k_{2D}\, t_{comm}) = 4\,(t_{start} + t_{comm}\, N/\sqrt{P})$

, respectively. Note that the computation time per node should be the same as in 1D case, but the communication cost is different because in 2D each node has to communicate 4 batches of k_{2D} values (in each direction). Again, for simplicity corrections at the boundary nodes were neglected assuming a large P.

1D decomposition will be more beneficial if

$T_{comp,1D} + T_{comm,1D} < T_{comp,2D} + T_{comm,2D}$.

Since $T_{comm,1D} = T_{comp,2D}$, the benefit can come only at the cost of faster data transactions, which is possible if

$t_{start} + N\, t_{comm} < 2(t_{start} + t_{comm}\, N/\sqrt{P})$,

which yields the ratio sought,

$t_{start}/t_{comm} > N\,(1 - 2/\sqrt{P})$.